

T3: «Kraków bez barier» — платформа доступных и разгруженных туристических маршрутов

T3 написано на русском для команды. Все тексты интерфейса, презентация и видео — **на польском** (требование организаторов).

0. Сроки и команда

- Команда: 3 человека.
- Этап 1 — прототип до 20:00 (сегодня):** только Flutter, без бэкенда. Данные берутся из Overpass/ORS и локальных JSON с пометкой «DANE PRZYKŁADOWE».
- Этап 2 — финал до 11:00 (завтра):** бэкенд на Java Spring Boot + PostGIS, ИИ-анализ фото, PDF до 10 слайдов, видео до 3 минут в открытом репозитории.

1. Проблема и цель

Людям на инвалидной или детской коляске метки «dostępne/niedostępne» мало. Им нужны конкретные данные: ступени, бордюры, покрытие, ширина двери, туалет, места для отдыха. Плюс нужно понимать, насколько эти данные свежие и надёжные. Отдельная проблема — туристические толпы: в переполненном месте на коляске проехать ещё труднее.

Цель: приложение, которое

- показывает детальную доступность мест и маршрутов с учётом **индивидуальных порогов** пользователя;
- строит маршрут, который **распределяет людей** по городу (обходит переполненные квадраты);
- через игровую механику (существа + ваучеры) **мотивирует людей картировать барьеры** и уходить в менее загруженные районы.

Соответствие критериям жюри

Критерий (вес)	Чем закрываем
Польза для группы, удобство (25%)	Профиль порогов, детальная карточка места, текстовый список вместо карты, маршрут без барьеров
Качество прототипа (20%)	Рабочий демо-сценарий на реальных данных OSM
Достоверность данных (15%)	У каждого факта есть источник, дата и уровень доверия; конфликтующие данные показываются явно; «brak danych ≠ dostępne»

Критерий (вес)	Чем закрываем
Внедрение и масштабирование (20%)	Адаптеры источников; новый город = bbox + конфиг; docker-compose
Бизнес-модель (20%)	B2B-бейдж, ваучеры по модели CPA, API/виджет, white-label, аналитика толп

2. Целевая группа и профиль потребностей

- **Пользователи инвалидных колясок** (ручных и электрических).
- **Родители с детскими колясками.**
- Приложение **не спрашивает об инвалидности**, только о предпочтениях (по требованию PDF). Пресеты: «Wózek inwalidzki», «Wózek dziecięcy», «Własne».

Параметр	Wózek inwalidzki	Wózek dziecięcy
Макс. ступеней	0	2
Макс. высота бордюра	3 см	6 см
Мин. ширина двери/прохода	80 см	60 см
Макс. уклон	6%	10%
Нужен доступный туалет	да	нет (нужен пеленальный столик — опционально)
Лавочки каждые N м	опц.	опц.
Избегать толп	да	да

Профиль хранится локально, на сервер уходит только в виде параметров запроса маршрута.

3. Пользовательские сценарии

Главный демо-сценарий:

1. Onboarding → выбор пресета «Wózek inwalidzki».
2. Поиск «Wawel» → карточка места: список барьеров и удобств, у каждого источник, дата и значок доверия.
3. «Zaplanuj trasę» из Rynek, Kazimierz, Wawel → маршрут обходит ступени и высокие бордюры, порядок остановок подстроен под толпу («Wawel teraz zatłoczony — przesunięty na 16:00»).
4. Переключение «Pokaż jako listę» → весь маршрут в текстовом виде (для скринридера).
5. На маршруте появляется существо → фото места → ИИ отмечает «4 schody, brak rampy, krawężnik >7 cm» → факт попадает на карту со статусом «AI · niezweryfikowane».
6. Очки → ваучер в кафе в малолюдном квадрате, таймер 2 ч.

Обязательные краевые сценарии (по PDF, раздел 6):

- **Конфликт данных:** OSM говорит `wheelchair=yes`, пользователь сообщает «3 schody». Показываем оба факта и предупреждение «Dane sprzeczne — sprawdź przed wizytą».

- **Нет данных:** серый статус «Brak danych o dostępności». Такое место **никогда** не засчитывается как доступное, при маршрутизации получает штраф.
- **Источник недоступен** (Overpass вернул 429 или таймаут): берём кэш, показываем баннер «Dane z dnia 03.10.2026, 14:20 — źródło chwilowo niedostępne».
- **Устаревшие данные** (старше 12 месяцев): статус «Nieaktualne» и кнопка «Potwierdź aktualność».

4. Функциональные требования по экранам

Приоритеты: **P0** — прототип к 20:00, **P1** — финал к 11:00, **P2** — roadmap (только в презентации).

4.1 Мара (главный экран)

- P0: карта `flutter_map` + тайлы OSM с атрибуцией «© OpenStreetMap contributors».
- P0: маркеры мест, цвет совпадает с итоговой оценкой для профиля (зелёный / жёлтый / красный / серый = brak danych). Каждый маркер дополнительно несёт **иконку и текст**, а не только цвет.
- P0: поиск места (по локальному списку POI из Overpass).
- P0: чипы-фильтры: Wózek, Unikaj tłumów, Toalety, Ławki.
- P1: слой толп (сетка 250×250 м, полупрозрачные квадраты).
- P1: существа на карте.
- P0: кнопка «Lista» — текстовая альтернатива карте.

4.2 Karta miejsca (детали)

- P0: блок «Barierę» (ступени, бордюр, порог, покрытие, уклон) и блок «Udogodnienia» (пандус, лифт, туалет, вход, ширина двери, лавочки, парковка для инвалидов).
- P0: у каждого факта есть значение, **источник, дата** («pobrano / potwierdzono»), **уровень доверия** (см. раздел 5).
- P0: итог для профиля: «Pasuje do Twoich potrzeb» / «Częściowo — 1 bariera» / «Nie pasuje» / «Za mało danych». Перечислено, **какие именно** параметры не прошли.
- P1: кнопки «Potwierdź» / «Zgłoś błąd» / «Dodaj zdjęcie».
- P1: текущая загруженность квадрата с источником оценки.

4.3 Trasa

- P0: маршрут из 2–5 точек через OpenRouteService, профиль `wheelchair`. Параметры профиля передаются в `restrictions`: `maximum_incline`, `maximum_sloped_kerb`, `minimum_width`. Покрытие (брусчатку) не учитываем.
- P0: список сегментов с предупреждениями («Odcinek 3: schody, 4 stopnie»).
- P1: **перебалансировка порядка остановок** по толпам:
 - стоимость = $\Sigma(\text{время пути}) + \lambda \cdot \Sigma(\text{crowd}(\text{cell}, \text{ETA}) \cdot \text{время в точке})$;
 - перебор всех перестановок при ≤ 6 точках, иначе жадный алгоритм + 2-opt;
 - пересчёт каждые 10 минут или по кнопке «Przelicz wg tłumów».

- P1: «rozproszenie» — разным пользователям с одинаковым запросом даём разные, но равноценные варианты: случайность в пределах +10% стоимости с учётом числа уже выданных маршрутов в квадрат.
- P2: групповые прогулки.

4.4 Złap / Zgłoś (камера)

- P1: фото → Java API → Kafka (photo.submitted) → **Python-сервис (FastAPI, компьютерное зрение) → Gemini API (Flash, structured output) → JSON**: {steps:int?, kerb_range:"0-3"|"3-7"|">7"|null, width_range:"<70"|"70-90"|">90"|null, ramp:bool?, handrail:bool?, obstacles:[...], difficulty:0-10, confidence:0-1}. Сантиметры по одному фото не измерить, поэтому ИИ возвращает **диапазоны**.
- Python-сервис до вызова Gemini (OpenCV, Pillow, imagehash):
 1. проверка качества (смазано или темно → «Zrób zdjęcie ponownie»);
 2. уменьшение до ~1280 px, удаление EXIF после чтения GPS и времени;
 3. perceptual hash против дубликатов (одно и то же место);
 4. размытие лиц и номеров (детектор OpenCV, P1/P2).
- P2: YOLO-детектор ступеней и бордюров как второе мнение. Если он и Gemini расходятся → «nierewne».
- P0 (прототип): ручная анкета с чекбоксами барьеров (fallback для режима без ИИ).
- Проверки: GPS ≤ 50 м от точки спавна; время в EXIF не старше 10 минут.
- Gemini free tier может использовать данные для обучения: для демо допустимо, в продакшене — платный тариф.

4.4.1 Как считаются метрики

Параметр	OSM (Overpass)	Фото + ИИ	Ручной отчёт
Ступени	highway=steps + step_count; у входа entrance + wheelchair, step_count	число видимых ступеней	«ile schodów»
Бордюр	kerb:height, либо тип: flush =0, lowered ≈2, rolled ≈4, raised ≈12 см (помечается «szacunek z typu»)	диапазон 0–3 / 3–7 / >7 см	диапазон
Ширина	width, door:width	<70 / 70–90 / >90 см	диапазон
Уклон	incline; если нет — расчёт по модели рельефа (NMT/LiDAR geoportal.gov.pl или MSIP): Δвысоты / длина, «Szacunek (model terenu)»	—	stromo / łagodnie / płasko
Удобства	toilets:wheelchair, ramp, elevator, amenity=bench, parking_space=disabled	пандус, поручень	чекбоксы

Оценка: по каждому параметру берём худший достоверный факт и сравниваем с порогом профиля → пройдено / на грани (±20%) / не пройдено / нет данных.

- Критичные параметры (ступени, бордюр, ширина) не пройдены → «Nie pasuje».
- Не пройден некритичный параметр (уклон) или что-то на грани → «Częściowo».
- Нет данных по критичному параметру → «Za mało danych».
- Всё пройдено → «Pasuje».

4.5 Kolekcja

- P1: сетка существ (common, rare, epic, legendary); заблокированные существа показывают район, где их можно найти.
- P1: «Odkryte miasto»: % посещённых квадратов в целом и по районам.

4.6 Nagrody

- P1: баланс очков, список ваучеров у партнёров (только с подтверждённой доступностью входа).
- P1: активация → QR + таймер 2 ч → после истечения ваучер «wygaść».

4.7 Profil

- P0: профиль потребностей (раздел 2).
- P1: статистика: сколько барьеров отмечено, сколько км пройдено, «% tłumu ominięty».
- P0: язык интерфейса PL (P2: EN, UA).

5. Модель достоверности данных

Каждый факт хранится в виде AccessibilityFact:

```
feature    (steps | kerb_height | door_width | ramp | elevator | toilet | bench | incline ...)
value
source     (OSM | MSIP | OTWARTE_DANE | OWNER | USER | AI | ESTIMATE)
source_ref (id объекта OSM / id отчёта / URL набора данных)
fetched_at, confirmed_at, confirmations_count, disputes_count
trust      (вычисляется)
```

Уровень	Условие	Отображение
 Potwierdzone	владелец объекта ИЛИ OSM + ≥2 независимых подтверждения ИЛИ ≥3 пользователя	зелёная галочка + «Potwierdzone (5 osób, 2 dni temu)»
 Dane otwarte	OSM / MSIP / Otwarte Dane без подтверждений	«OpenStreetMap, edycja 12.2025»
 Zgłoszone	1 отчёт пользователя	«Zgłoszenie użytkownika — niezweryfikowane»
 AI	только ИИ-анализ	«Ocena AI — niezweryfikowane»
 Sprzeczne	есть противоречащие факты	показываем все + предупреждение

Уровень	Условие	Отображение
 Nieaktualne	старше 12 месяцев	серый текст + «Potwierdź»
☐ Brak danych	фактов нет	«Brak danych» (≠ dostępne)

Исправление ошибок: «Zgłoś błąd» создаёт спорный отчёт (dispute). При ≥ 2 спорах значение понижается до «Sprzeczne». Новые отчёты пользователей можно отправлять обратно в OSM в виде заметок (OSM Notes, P2).

Итоговая оценка места или сегмента для профиля = худший из **достоверных** фактов по каждому параметру профиля. Если для критичного параметра данных нет, результат «Za mało danych», а не «Pasuje».

6. Толпы (Crowd model)

Город делится на сетку 250×250 м (в Кракове около 5–6 тыс. квадратов; для MVP — центр, около 400).

$\text{crowd}(\text{cell}, t) \in [0..1]$ складывается из трёх слоёв:

- Базовая оценка (всегда есть):** $\text{base} = \text{norm}(\sum \text{веса POI в квадрате}) \times \text{hour_profile}[\text{type}][\text{dow}][\text{hour}]$. Веса POI из Overpass: `tourism=attraction 5, historic=* 3, amenity=restaurant|cafe|bar 1, public_transport=stop_position 1`. Часовые профили — эвристика, по типам места (музей, ресторан, площадь). Помечается «Szacunek».
- Опрос «Jak tłoczno?»** — одно касание (● / ● / ●) при входе в квадрат или в карточке места. Вес ответа затухает экспоненциально, полураспад 30 минут. За ответ +5 очков, не чаще одного раза в квадрат за 30 минут.
- Активные пользователи приложения** в квадрате (анонимно, агрегировано, без истории перемещений).

Итог: $\text{crowd} = w_1 \cdot \text{base} + w_2 \cdot \text{survey} + w_3 \cdot \text{live}$. Веса зависят от свежести и количества данных. В UI всегда виден источник: «Szacunek (OSM + pora dnia)» или «12 osób zgłosiło, ostatnio 8 min temu».

Источники, которые проверить в первую очередь (из списка PDF): otwartedane.um.krakow.pl и MSIP — счётчики пешеходов и велосипедистов, календарь мероприятий, загрузка паркингов. Найдётся что-то — добавляем четвёртым слоем-адаптером. **Не используем:** Google Popular Times, Yandex, Apple Maps, Snap Map — нет публичного API или запрещено ToS. В PDF, стр. 6: «samo opublikowanie... nie oznacza, że wolno ją automatycznie pobierać». BestTime.app — платный адаптер в roadmap (P2).

7. Игровая механика

7.1 Спавн существ

- Квадрат получает $\text{difficulty}(\text{cell}) \in [0..1]$ из OSM: доля сегментов со ступенями, высоким бордюром, крутым уклоном; объекты с `wheelchair=no/limited`.
- **Потенциально трудные места** определяются тремя способами:
 1. OSM-теги (ступени, `kerb=raised`, `incline>6%`, `wheelchair=no`);
 2. **дыры в данных**: POI без тега `wheelchair` и переходы без `kerb` — там картировать полезнее всего;
 3. отчёты пользователей и приманки.
- Вероятность спавна в точке пропорциональна difficulty + бонус за «неизвестность» (`brak danych`).
- **Редкость**: $\text{rarity_score} = 0.5 \cdot \text{difficulty} + 0.3 \cdot \text{unexplored} + 0.2 \cdot \text{low_crowd}$, где $\text{unexplored} = 1 / (1 + \text{число фото в радиусе } 30 \text{ м})$. Пороги: <0.35 common, <0.6 rare, <0.8 epic, иначе legendary. Повторное фото того же места **понижает** редкость.
- Нагрузка на точку ограничена: после N подтверждений существа там больше не появляются, место считается закартированным.

7.2 Приманка (Przynęta)

- Стоит 50 очков. Пользователь ставит её там, где, по его мнению, есть барьер. Шанс спавна в радиусе 50 м растёт в 3 раза на 24 часа. Если кто-то сделает там подтверждённое фото, автор приманки получает +30% очков.

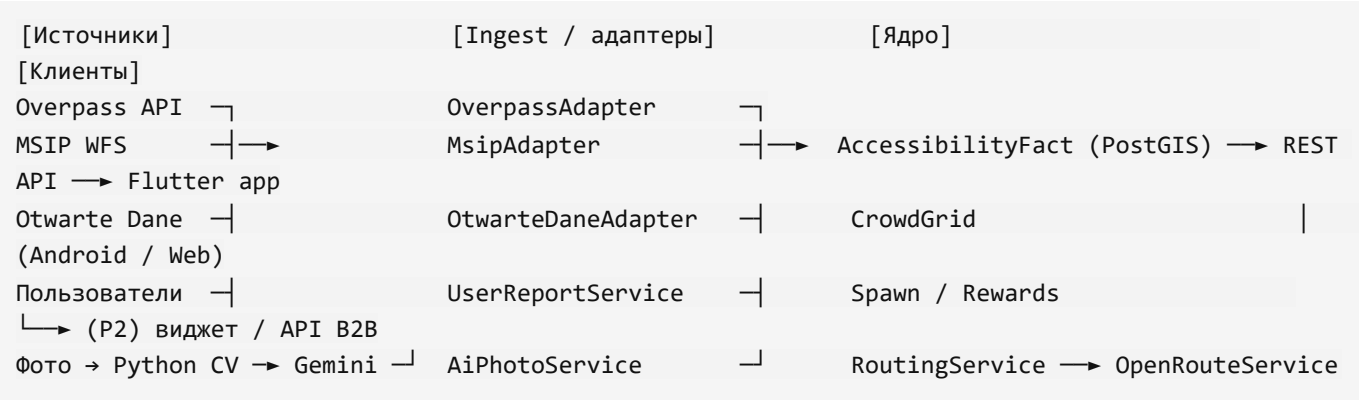
7.3 Антиабюз

- Кулдаун 30 м / 1 спавн в 15 минут на пользователя; GPS-проверка; EXIF и время съёмки; perceptual hash; лимит фото в сутки; детект «телепортации» (скорость между событиями больше 30 км/ч — подозрительно).
- Если ИИ ответил с $\text{confidence} < 0.5$, очки делятся пополам до подтверждения.

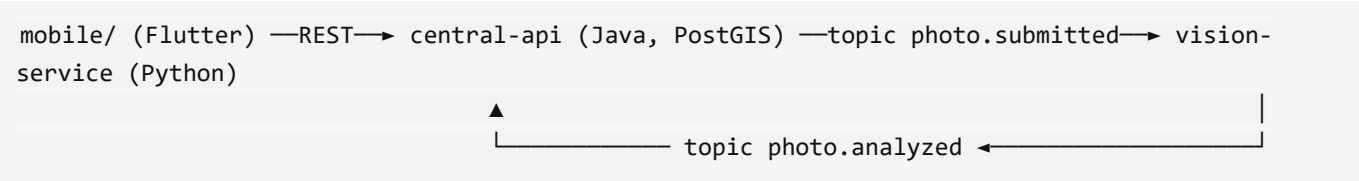
8. Очки и ваучеры

- Очки не сгорают. Common 10, rare 25, epic 60, legendary 150, опрос толпы 5, подтверждение чужого факта 15.
- **Ваучер** покупается за очки в любой момент. **После активации** действует 2 часа (подталкивает пойти сейчас и туда).
- **Квота ваучеров на точку** = $\text{base_quota} \times (1 - \text{crowd}(\text{cell}))$, пересчёт каждые 15 минут. Где пусто — больше ваучеров, на Rynek в пиковое время — почти ноль.
- Партнёр задаёт скидку, base_quota и часы. Попасть в программу можно только с **подтверждённой доступностью входа** (проверяется по разделу 5). Это стимул для заведений улучшать доступность.
- Прототип: партнёры и ваучеры — демо-данные с пометкой «DANE PRZYKŁADOWE».

9. Архитектура



Микросервисы (монорепо):



Сервис	Стек	Ответственность
mobile	Flutter	UI, профиль потребностей (локально), карта, камера
central-api	Java 21, Spring Boot 3, PostGIS	REST для клиента, адаптеры источников, достоверность, маршруты (прокси ORS), толпы, игра, ваучеры
vision-service	Python 3.12, FastAPI, OpenCV, google-gemai	анализ фото: предобработка + Gemini
брокер	Redpanda (Kafka API)	асинхронная связь central-api ↔ vision
БД	PostgreSQL 16 + PostGIS	только у central-api

- **Отделение данных от представления** (требование PDF): адаптеры реализуют интерфейс `SourceAdapter { fetch(bbox) → List<AccessibilityFact> }`. Новый источник = новый класс + запись в конфиге.
- **Новый город** = запись `City{name, bbox, timezone, grid_size, enabled_adapters}` + запуск импорта. Код не меняется.
- **Новая категория мест** = маппинг тегов OSM → feature в YAML.
- **Обновление:** Overpass — раз в сутки (diff по timestamp), MSIP и Otwarte Dane — раз в неделю, толпы — каждые 5 минут.
- **Недоступность источника:** 3 ретрая с экспоненциальной задержкой, затем последний кэш + флаг stale в API → баннер в UI.

9.1 Этап 1 (прототип, без бэкенда)

- Flutter обращается к Overpass (запросы по bbox центра, ответ кэшируется в `shared_preferences/файл`) и к ORS (ключ в `--dart-define`, только для демо).
- Снимок данных Overpass для центра лежит в `assets/data/osm_center.json` — fallback, если API недоступен.
- Демо-толпы, партнёры и существа — в `assets/demo/*.json`, в UI бейдж «DANE PRZYKŁADOWE».
- Абстракция `Repository` с реализациями `LocalRepository` (этап 1) и `ApiRepository` (этап 2), UI не меняется.

9.2 Этап 2: бэкенд Java Spring Boot + PostGIS

- Java 21, Spring Boot 3, Spring Web, Spring Data JPA + **Hibernate Spatial** (JTS, SRID 4326), Flyway, Spring Security (анонимный JWT: устройство получает токен без регистрации), `@Scheduled` (импорт и пересчёт сетки), Bean Validation, springdoc-openapi (Swagger).
- Хранение фото: локальный том, P2 — MinIO/S3.
- Анализ фото — отдельный сервис `vision-service` (Python 3.12, FastAPI, OpenCV, imagehash, google-generativeai, aiokafka). Наружу не открыт, ключ Gemini хранится только в нём.
- Связь через Kafka API (Redpanda, один узел):**
 - Flutter → POST `/api/v1/catches` (multipart). `central-api` сохраняет фото в общий том `/photos`, создаёт `catch` со статусом `PENDING` и отвечает 202 `{catchId}`.
 - `central-api` публикует в топик `photo.submitted: {catchId, photoPath, lat, lng, spawnId, submittedAt}`. **Байты фото в Kafka не передаём**, только путь.
 - `vision-service` читает событие, выполняет предобработку и вызов Gemini, публикует в `photo.analyzed: {catchId, status: OK|REJECTED|FAILED, result{...раздел 4.4...}, reason?}`.
 - `central-api` читает `photo.analyzed`, обновляет `catch`, создаёт `AccessibilityFact` (`source=AI`), начисляет очки.
 - Flutter опрашивает GET `/api/v1/catches/{id}` раз в 1–2 с (P2: SSE).
- Ключ сообщения = `catchId` (порядок в рамках одного фото). Обработка идемпотентна: повтор события не начисляет очки повторно. Ошибка → 3 ретрая, затем топик `photo.analyzed.dlq` и статус `FAILED` («AI niedostępne, użyj ankiety»).
- `docker-compose.yml`: `postgis/postgis:16-3.4`, `redpandadata/redpanda` (≈200 МБ ОЗУ), `central-api`, `vision-service`, общий том `photos`. Деплой: VPS (Hetzner/Oracle Free) одним `docker compose up`. Managed Kafka на бесплатных PaaS обычно недоступна. **Прогреть инстанс перед демо.**
- CORS для Flutter Web.

Подводные камни, которые учесть:

- SRID: везде 4326, `ST_DWithin` по `geography`, иначе расстояния будут в градусах.
- GIST-индексы на всех геометриях.
- Rate limit у Overpass: импорт идёт одним запросом по bbox, а не по запросу на каждого пользователя.

- Лимиты ORS на бесплатном ключе (около 2000 запросов в сутки): маршруты кэшировать.
- Размер фото: сжимать на клиенте до ~1280 px и JPEG 80%.

9.3 Модель данных (PostGIS)

```
city(id, name, bbox geometry(Polygon), tz, grid_size_m)
place(id, city_id, osm_id, name, category, geom geometry(Point), updated_at)
accessibility_fact(id, place_id NULL, segment_id NULL, feature, value jsonb, source,
source_ref,
                fetched_at, confirmed_at, confirmations, disputes, trust)
segment(id, osm_way_id, geom geometry(LineString), incline, width, kerb_height, steps bool)
grid_cell(id, city_id, geom geometry(Polygon), poi_weight, difficulty, explored_count)
crowd_report(id, cell_id, level smallint, created_at, device_hash)
crowd_snapshot(cell_id, ts, crowd, source_mix jsonb)
creature_spawn(id, cell_id, geom, rarity, species, expires_at, bait_id NULL)
catch(id, user_id, spawn_id, photo_url, ai_result jsonb, ai_confidence, phash, created_at,
status)
bait(id, user_id, geom, expires_at)
app_user(id, anon_token_hash, points, created_at)          -- без PII и данных о здоровье
partner(id, place_id, name, verified_access bool)
voucher_offer(id, partner_id, discount, base_quota, cost_points, valid_hours)
voucher(id, offer_id, user_id, activated_at, expires_at, redeemed_at)
```

9.4 API (REST, /api/v1)

GET /places?bbox=&category=&profile=	места + итоговая оценка для профиля
GET /places/{id}	детали + все факты с источниками
POST /places/{id}/facts	отчёт пользователя
POST /facts/{id}/confirm /dispute	
POST /routes {points[], profile, avoidCrowds}	маршрут + сегменты + предупреждения + порядок
GET /crowd?bbox=&at=	сетка толп + источник
POST /crowd/report {lat,lng,level}	
GET /spawns?bbox=	
POST /catches (multipart photo, spawnId, lat, lng) → 202 {catchId}	
GET /catches/{id}	статус PENDING/OK/REJECTED/FAILED + ai_result,
points	
POST /baits	
GET /rewards/offers?bbox=	POST /vouchers/{offerId}/activate
GET /me (очки, статистика, % исследованности)	
GET /health/sources	статус источников (для баннера)

9.5 Промпт ИИ (сервер)

Модель: Gemini Flash (актуальная версия, режим `response_schema`), вызывается из Python-сервиса `vision`. Системный промпт задаёт роль аудитора доступности и **строгий JSON** из раздела 4.4. Правила: «если не видно — null, не выдумывай», confidence оценивается честно. Ответ валидируется по схеме; если невалиден — 1 ретрай, затем статус «Al niedostępne, użyj ankiety».

10. Flutter-клиент

- Пакеты: `flutter_map`, `latlong2`, `geolocator`, `image_picker` (или `camera`), `http/dio`, `flutter_riverpod`, `shared_preferences`, `qr_flutter`, `go_router`.
- Структура `lib/`: `core/` (тема, константы, `a11y`), `data/` (`models`, `repositories`: `local/api`, `sources`: `overpass`, `ors`), `domain/` (`profile matching`, `crowd`, `route ordering`), `features/{map,place,route,catch,collection,rewards,profile,onboarding}/`.
- Структура экранов берётся из `krakow-bez-barrier-interface-draft.html`, но **стиль — светлый, мятно-белый** (тёмную тему пока не делаем). NFT убрать.
- **Палитра:**

Токен	Цвет	Где
background	#FFFFFF	фон экранов
surface	#F4FBF8	карточки, нижние панели
mint-100	#E3F6EE	чипы, подложки, выбранные элементы
mint-300	#9FE0C6	декор, слой «мало людей»
mint-500	#3EB489	иконки, линия маршрута, крупная графика
primary	#0E7A5A	кнопки (белый текст), ссылки, активные табы
border	#DCEFE7	обводки 1px
text	#0F1F1A	основной текст
text-muted	#4A5B55	второстепенный текст
ok / warn / bad / unknown	#0E7A5A / #8A5A00 (фон #FFF4D6) / #B42318 (фон #FDECEA) / #5F6B76	статусы доступности

- Стиль: шрифт Manrope, скругления 16–20 px, мягкие тени (`0 2px 12px rgba(14,122,90,.08)`), много воздуха, иконки линейные.
- Контраст текста $\geq 4.5:1$ (`mint-500` не использовать для мелкого текста), зона касания ≥ 48 dp.
- **Доступность (WCAG 2.2 AA):** `Semantics` у всех маркеров и кнопок; фокус-порядок и работа с клавиатуры (Flutter Web); текстовый список вместо карты; статусы передаются текстом и иконкой, а не только цветом; поддержка увеличения шрифта до 200%; без анимаций при `MediaQuery.disableAnimations`.

11. Приватность и безопасность

- Анонимная учётка (токен устройства), без email и **без вопросов о здоровье и инвалидности**.
- Профиль потребностей хранится только на устройстве.

- GPS используется только в момент события (фото или опрос), треки не храним. Толпы агрегируются по квадратам, минимум 3 пользователя, иначе не показываем.
- EXIF удаляется после проверки; P2 — размытие лиц и номеров.
- Только HTTPS; rate limiting на API; модерация отчётов (жалоба → скрытие при ≥3 жалобах).
- Соответствие RODO: политика приватности, удаление данных по запросу.

12. Лицензии и зависимости

Компонент	Лицензия / условия	Риск
OpenStreetMap / Overpass	ODbL, атрибуция обязательна; производная база — share-alike	лимиты публичного инстанса → свой Overpass в P2
Тайлы OSM	tile usage policy (не для продакшен-нагрузки)	P2: MapTiler или свой сервер тайлов
OpenRouteService	бесплатный ключ с лимитами; open-source, можно self-host	self-host в P2
Otwarte Dane / MSIP	проверить для каждого набора (обычно CC BY)	указать в UI
Gemini API	free tier (данные могут идти в обучение) / платно в продакшене	fallback на анкету
OpenCV, imagehash, FastAPI	Apache 2.0 / BSD / MIT	—
Flutter, Spring, PostGIS	BSD / Apache 2.0 / GPL (PostGIS — сервер, не распространяем)	—

13. Бизнес-модель

1. **B2B для заведений:** бейдж «Zweryfikowana dostępność» + профиль на карте + участие в ваучерной программе. Подписка 49–149 zł в месяц.
 2. **Ваучеры — CPA:** партнёр платит 1–3 zł за погашенный ваучер. Ценность для партнёра — гости в непиковые часы.
 3. **API и виджет доступности** для отелей, организаторов событий, систем бронирования: «доступность вокруг объекта», маршрут от отеля. Тарифы по числу запросов.
 4. **White-label для городов и DMO:** аналитика толп и карта барьеров (где вкладываться в ремонт) — лицензия на год.
 5. **Организаторы мероприятий:** временный слой доступности для события.
- Себестоимость MVP: хостинг около 50–100 zł в месяц, ИИ около 0,01–0,03 zł за фото.

14. Внедрение и сопровождение

- **Ответственный:** команда в форме стартапа (sp. z o.o.). Хостинг у облачного провайдера в ЕС, данные не на инфраструктуре UMK.
- **Обновление данных:** автоматические адаптеры + сообщество. Город ничего не поддерживает вручную.
- **Обработка отчётов:** модерация силами сообщества + админ-панель (P2).
- **Новый город:** добавить bbox и конфиг → импорт OSM → часовые профили по умолчанию → привлечь 5–10 партнёров. Около 1 дня технической работы.
- **Roadmap:** P2 — свой Overpass и ORS, BestTime, OSM Notes, iOS, EN/UA, админ-панель, размытие лиц, офлайн-режим.

15. Ограничения прототипа (честно указать в презентации)

- Толпы — эвристика + демо-данные, а не реальные замеры.
- Партнёры и ваучеры — демо.
- ИИ может ошибаться, поэтому всё помечается «niezweryfikowane».
- Покрытие только центра Кракова.
- WCAG проверен вручную (TalkBack или NVDA + клавиатура + контраст), без аудита.

16. План работ

Роли: А — Flutter UI; В — данные, маршруты, затем Java-бэкенд; С — игра, ИИ, презентация и видео.

15:00–20:00 (прототип):

- А: каркас приложения, навигация, тема, Mapa + Karta miejsca + Lista + Profil (пресеты).
- В: Overpass-запрос для центра → парсер в `AccessibilityFact` + снимок в `assets`; ORS-маршрут с профилем; сопоставление с профилем; бейджи доверия.
- С: демо-JSON (толпы, существа, партнёры), экран `Złap` с ручной анкетой, `Kolekcja` и `Nagrody` на демо-данных.
- 19:30 — сборка Web + APK, прогон главного сценария, отправка.

20:00–11:00 (финал):

- В: Spring Boot + PostGIS (сущности, Flyway, импорт Overpass, API из 9.4), деплой.
- С: `vision-service` (OpenCV + Gemini, Kafka consumer/producer), спавн и редкость, ваучеры с таймером и квотой.
- А: переключение на `ApiRepository`, слой толп, перебалансировка маршрута, проверка a11y.
- 07:00–10:30: PDF (≤10 слайдов, на польском), видео (≤3 мин) → открытый репозиторий, README с лицензиями и источниками.

17. Чек-лист сдачи (по PDF)

- ☐ Описание решения и проблемы
- ☐ Прототип / демо
- ☐ Целевая группа и способ использования
- ☐ Источники данных + как оцениваем актуальность и достоверность
- ☐ Бизнес-модель и развитие
- ☐ PDF ≤ 10 слайдов (PL)
- ☐ Видео ≤ 3 мин в открытом репозитории
- ☐ Демо: конфликт данных / нет данных / недоступный источник
- ☐ Демо-данные явно помечены
- ☐ Проверка доступности: клавиатура, скринридер, контраст, текстовая альтернатива карте + список ограничений
- ☐ План перехода к сервису и запуска в другом городе